

# CS310 NLP Assignment 1 实验报告

---

## 1. 实验任务与文件说明

本实验任务为中文幽默检测二分类，目标是判断句子是否具有幽默性（标签 0/1）。

本报告结合以下 3 个文件完成：

1. `A1_nn.ipynb`：完整实验代码（数据处理、建模、训练、评估）
2. `train.jsonl`：训练数据
3. `test.jsonl`：测试数据

实验环境与固定设置：

- 框架：PyTorch
- 随机种子：310
- 设备：CPU

数据规模：

- 训练集：12677
- 验证集：1267（由训练集按 10% 划分）
- 测试集：651

## 2. 实验过程

### 2.1 数据读取与预处理

- 从 `train.jsonl`、`test.jsonl` 逐行读取样本。
- 每条样本保留字段：`sentence`、`label`、`id`。
- 标签处理方式：取 `label[0]` 并转为整数。

### 2.2 两种分词方案设计

为满足作业中“比较至少两种 tokenizer”的要求，实验实现了两套方案：

#### 1. `basic_char`

- 仅保留中文字符，按单字切分。
- 丢弃英文、数字和标点。

#### 2. `advanced`

- 使用正则先切分出英文串、数字串、中文串和标点。
- 对中文串再使用“短语词典 + 最大匹配”做进一步分词。
- 短语词典来自训练集统计，设置 `min_freq=8`，`max_word_len=4`。

词表构建结果：

- `basic_char` 词表大小：2687

- **advanced** 词表大小 : 7345

2.3 模型结构

模型采用 **EmbeddingBag + MLP** :

- Embedding 层 : `nn.EmbeddingBag(vocab_size, 128, mode="mean")`
- 分类头 : 两层隐藏层全连接网络
  - **128 -> 128 -> 64 -> 2**
  - 激活函数 : ReLU
  - Dropout : 0.25

训练参数 :

- 损失函数 : CrossEntropyLoss
- 优化器 : Adam (学习率 1e-3)
- 训练轮数 : 10 epochs
- 学习率调度 : ReduceLROnPlateau (依据验证集 F1)
- 梯度裁剪 : `max_norm=1.0`

参数规模 :

- 可训练参数总量 : 965058

2.4 训练与模型选择策略

- 在每个 epoch 结束后, 在验证集计算 **accuracy / precision / recall / f1**。
- 使用验证集 F1 最高时的模型参数作为该 tokenizer 的最佳模型。
- 最终在测试集报告结果, 并以测试集 F1 最高者作为本次实验最佳方案。

3. 实验结果

```
Running experiment: basic_char (single Chinese characters only; drop English, digits, and punctuations)
[basic_char] epoch 01 | train loss 0.6765 | train acc 0.6682 | valid acc 0.7024 | valid precision 0.3939 | valid recall 0.0351 | valid f1 0.0645
[basic_char] epoch 02 | train loss 0.5922 | train acc 0.7069 | valid acc 0.7064 | valid precision 0.4545 | valid recall 0.0270 | valid f1 0.0510
[basic_char] epoch 03 | train loss 0.5701 | train acc 0.7138 | valid acc 0.7072 | valid precision 0.4800 | valid recall 0.0324 | valid f1 0.0608
[basic_char] epoch 04 | train loss 0.5492 | train acc 0.7223 | valid acc 0.7056 | valid precision 0.4651 | valid recall 0.0541 | valid f1 0.0969
[basic_char] epoch 05 | train loss 0.5397 | train acc 0.7292 | valid acc 0.7103 | valid precision 0.5306 | valid recall 0.0703 | valid f1 0.1241
[basic_char] epoch 06 | train loss 0.5280 | train acc 0.7355 | valid acc 0.7096 | valid precision 0.5135 | valid recall 0.1027 | valid f1 0.1712
[basic_char] epoch 07 | train loss 0.5148 | train acc 0.7421 | valid acc 0.7127 | valid precision 0.5385 | valid recall 0.1135 | valid f1 0.1875
[basic_char] epoch 08 | train loss 0.4981 | train acc 0.7528 | valid acc 0.7056 | valid precision 0.4860 | valid recall 0.1405 | valid f1 0.2180
[basic_char] epoch 09 | train loss 0.4867 | train acc 0.7658 | valid acc 0.6993 | valid precision 0.4615 | valid recall 0.1784 | valid f1 0.2573
[basic_char] epoch 10 | train loss 0.4652 | train acc 0.7786 | valid acc 0.6969 | valid precision 0.4545 | valid recall 0.1892 | valid f1 0.2672
=====
Running experiment: advanced (regex patterns plus phrase lexicon segmentation for Chinese spans)
[advanced] epoch 01 | train loss 0.7341 | train acc 0.6476 | valid acc 0.7080 | valid precision 0.5000 | valid recall 0.0189 | valid f1 0.0365
[advanced] epoch 02 | train loss 0.5870 | train acc 0.7067 | valid acc 0.7072 | valid precision 0.4444 | valid recall 0.0108 | valid f1 0.0211
[advanced] epoch 03 | train loss 0.5451 | train acc 0.7273 | valid acc 0.7127 | valid precision 0.6000 | valid recall 0.0486 | valid f1 0.0900
[advanced] epoch 04 | train loss 0.5030 | train acc 0.7524 | valid acc 0.7017 | valid precision 0.4565 | valid recall 0.1135 | valid f1 0.1818
[advanced] epoch 05 | train loss 0.4515 | train acc 0.7872 | valid acc 0.6993 | valid precision 0.4476 | valid recall 0.1270 | valid f1 0.1979
[advanced] epoch 06 | train loss 0.3896 | train acc 0.8237 | valid acc 0.6859 | valid precision 0.4255 | valid recall 0.2162 | valid f1 0.2867
[advanced] epoch 07 | train loss 0.3188 | train acc 0.8619 | valid acc 0.6898 | valid precision 0.4422 | valid recall 0.2378 | valid f1 0.3093
[advanced] epoch 08 | train loss 0.2555 | train acc 0.8959 | valid acc 0.6780 | valid precision 0.4317 | valid recall 0.3243 | valid f1 0.3704
[advanced] epoch 09 | train loss 0.1986 | train acc 0.9207 | valid acc 0.6859 | valid precision 0.4421 | valid recall 0.2892 | valid f1 0.3497
[advanced] epoch 10 | train loss 0.1546 | train acc 0.9416 | valid acc 0.6772 | valid precision 0.4247 | valid recall 0.2973 | valid f1 0.3498
...
3) The average of four metrics (acc/precision/recall/f1) also favors advanced, which means overall quality is better, not just one single metric.

Best experiment (by F1): advanced
Model checkpoint saved to: e:\CS310-Natural-Language-Processing\coding assignment\A1\best_advanced_model.pth
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

测试集指标如下 :

| tokenizer  | vocab_size | accuracy | precision | recall | f1     | loss   | tp | tn  | fp | fn  |
|------------|------------|----------|-----------|--------|--------|--------|----|-----|----|-----|
| basic_char | 2687       | 0.7312   | 0.4706    | 0.2353 | 0.3137 | 0.5587 | 40 | 436 | 45 | 130 |
| advanced   | 7345       | 0.7066   | 0.4305    | 0.3824 | 0.4050 | 0.6854 | 65 | 395 | 86 | 105 |

最佳模型（按 F1）：

- `advanced` (F1 = 0.4050)
- 模型权重保存为：`best_advanced_model.pth`

4. 结果分析

1. 从 Accuracy 看, `basic_char` 更高 (0.7312 > 0.7066) 。
2. 从 F1 看, `advanced` 明显更优 (0.4050 > 0.3137) 。
3. `advanced` 的 Recall 更高 (0.3824 > 0.2353) , 说明其对正类（幽默句）识别更充分。
4. `basic_char` 的策略更保守, 负类判定更稳定, 因此准确率较高, 但漏检正类较多 (FN=130) 。
5. 综合 precision 与 recall 的平衡需求, 本实验最终选择 `advanced` 作为最佳方案。